

C Program for Breadth First Search or BFS for a Graph

Last Updated : 09 Nov, 2023



The Breadth First Search (BFS) algorithm is used to search a graph data structure for a node that meets a set of criteria. It starts at the root of the graph and visits all nodes at the current depth level before moving on to the nodes at the next depth level.

Relation between BFS for Graph and Tree traversal:

[Breadth-First Traversal \(or Search\)](#) for a graph is similar to the [Breadth-First Traversal of a tree](#).

The only catch here is, that, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we divide the vertices into two categories:

- Visited and
- Not visited.

A boolean visited array is used to mark the visited vertices. For simplicity, it is assumed that all vertices are reachable from the starting vertex. BFS uses a [queue data structure](#) for traversal.

How does BFS work?

Starting from the root, all the nodes at a particular level are visited first and then the nodes of the next level are traversed till all the nodes are visited.

To do this a queue is used. All the adjacent unvisited nodes of the current level are pushed into the queue and the nodes of the current level are marked visited and popped from the queue.

Below is the implementation of the above approach:

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
```



```

#define MAX_VERTICES 50

// This struct represents a directed graph using
// adjacency list representation
typedef struct Graph_t {

    // No. of vertices
    int V;
    bool adj[MAX_VERTICES][MAX_VERTICES];
} Graph;

// Constructor
Graph* Graph_create(int V)
{
    Graph* g = malloc(sizeof(Graph));
    g->V = V;

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            g->adj[i][j] = false;
        }
    }

    return g;
}

// Destructor
void Graph_destroy(Graph* g) { free(g); }

// Function to add an edge to graph
void Graph_addEdge(Graph* g, int v, int w)
{
    // Add w to v's list.
    g->adj[v][w] = true;
}

// Prints BFS traversal from a given source s
void Graph_BFS(Graph* g, int s)
{
    // Mark all the vertices as not visited
    bool visited[MAX_VERTICES];

```

```

for (int i = 0; i < g->V; i++) {
    visited[i] = false;
}

// Create a queue for BFS
int queue[MAX_VERTICES];
int front = 0, rear = 0;

// Mark the current node as visited and enqueue it
visited[s] = true;
queue[rear++] = s;

while (front != rear) {

    // Dequeue a vertex from queue and print it
    s = queue[front++];
    printf("%d ", s);

    // Get all adjacent vertices of the dequeued
    // vertex s.
    // If an adjacent has not been visited,
    // then mark it visited and enqueue it
    for (int adjacent = 0; adjacent < g->V;
        adjacent++) {
        if (g->adj[s][adjacent] && !visited[adjacent]) {
            visited[adjacent] = true;
            queue[rear++] = adjacent;
        }
    }
}

// Driver code
int main()
{
    // Create a graph
    Graph* g = Graph_create(4);
    Graph_addEdge(g, 0, 1);
    Graph_addEdge(g, 0, 2);
    Graph_addEdge(g, 1, 2);
    Graph_addEdge(g, 2, 0);
    Graph_addEdge(g, 2, 3);
}

```

```
Graph_addEdge(g, 3, 3);

printf("Following is Breadth First Traversal "
      "(starting from vertex 2) \n");
Graph_BFS(g, 2);
Graph_destroy(g);

return 0;
}
```

Output

```
Following is Breadth First Traversal (starting from vertex 2)
2 0 3 1
```

Time Complexity: $O(V+E)$, where V is the number of nodes and E is the number of edges.

Auxiliary Space: $O(V)$

Please refer complete article on [Breadth First Search or BFS for a Graph](#) for more details!

 Comment

More info 

Advertise with us

Next Article 

Java Program for Breadth First Search
or BFS for a Graph

Similar Reads

Breadth First Search or BFS for a Graph

Given a undirected graph represented by an adjacency list adj, where each adj[i] represents the list of vertices connected to vertex i. Perform a Breadth First Search (BFS) traversal starting from vertex 0,...

 15+ min read

BFS in different language

[C Program for Breadth First Search or BFS for a Graph](#)

The Breadth First Search (BFS) algorithm is used to search a graph data structure for a node that meets a set of criteria. It starts at the root of the graph and visits all nodes at the current depth level before...

🕒 3 min read

Java Program for Breadth First Search or BFS for a Graph

The Breadth First Search (BFS) algorithm is used to search a graph data structure for a node that meets a set of criteria. It starts at the root of the graph and visits all nodes at the current depth level before...

🕒 3 min read

Breadth First Search or BFS for a Graph in Python

Breadth First Search (BFS) is a fundamental graph traversal algorithm. It begins with a node, then first traverses all its adjacent nodes. Once all adjacent are visited, then their adjacent are traversed. BFS is...

🕒 4 min read

BFS for Disconnected Graph

In the previous post, BFS only with a particular vertex is performed i.e. it is assumed that all vertices are reachable from the starting vertex. But in the case of a disconnected graph or any vertex that is...

🕒 14 min read

Applications, Advantages and Disadvantages of Breadth First Search (BFS)

We have earlier discussed Breadth First Traversal Algorithm for Graphs. Here in this article, we will see the applications, advantages, and disadvantages of the Breadth First Search. Applications of Breadth...

🕒 4 min read

Breadth First Traversal (BFS) on a 2D array

Given a matrix of size M x N consisting of integers, the task is to print the matrix elements using Breadth-First Search traversal. Examples: Input: `grid[][] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, ...}}`

🕒 9 min read

0-1 BFS (Shortest Path in a Binary Weight Graph)

Given an undirected graph where every edge has a weight as either 0 or 1. The task is to find the shortest path from the source vertex to all other vertices in the graph. Example: Input: Source Vertex = 0 and...

🕒 10 min read

Variations of BFS implementations



Easy problems on BFS



Intermediate problems on BFS



Hard Problems on BFS

